

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a4a649f3c9bc787ab.jsonl`  
- SHA-256: `b80368471904e7c76966d3b868bb507074c2f7bdc00a49fc7e39bb0969a3edc3`  
- Source modified: `2026-06-13T19:14:49+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:13:53.213Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"rally_gate.gate_windows recomputes the play-axis/net estimate over the WHOLE trajectory once per candidate window (O(W*P log P))","severity":"low","file":"backend/pipeline/segmenters/fusion_hybrid.py:77 (and rally_gate.py:91-105)","detail":"_enrich_candidate_stats calls rally_gate.gate_windows(points, [(cw['start'],cw['end'])], fps, min_crossings=0) once per window (driven by the per-window list comprehension in trajectory_hybrid.py:203-206). gate_windows internally calls estimate_play_axis(points) every call, which sorts ALL visible xs and ys (two _spread() sorts, each O(P log P)) and _median over all points, plus _points_in scans all P points ? even though axis/net are window-independent and explicitly documented as 'estimated once over the whole trajectory (stable)'. So for W windows over a P-point trajectory this is O(W * P log P) of redundant work. It is pure-Python and GPU-free, so not catastrophic, but wasteful on long videos with many candidates.","fix":"Estimate axis/net/deadband once per video (call estimate_play_axis(points) in the segmenter or precompute), then either count crossings per window directly via count_net_crossings, or call gate_windows(points, all_windows, ...) a single time and index the result by window ? instead of one whole-trajectory re-estimate per window."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:56.046Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:13:57.011Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).  
2  
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,  
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
```

```

5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9      GPU-free, computed from the trajectory we already have); and, only when the person cue
10     is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11     (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12     fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13     never imports torch / touches the GPU.
14     - ``_select_for_handoff`` ? optional served Gate A/B: when
15     ``indexing.fusion.min_crossings``
16     (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17     are
18     dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19     **Persisted candidates remain the full superset** regardless, so the offline
20     experiment
21     harness can re-score any variant.
22
23     Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24     but
25     nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
26     seeds
27     and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
28     default
29     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30     through the
31     experiment harness (EXPERIMENT_HARNESS.md), never silently.
32     """
33     from typing import Any, Dict, List, Optional, Tuple
34
35     from backend.pipeline.segmenters.base import segmenter_registry
36     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37     from backend.pipeline.detectors.base import DetectorRunner
38     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40     from backend.pipeline.detectors import rally_gate
41     from backend.pipeline.detectors import fusion_features as ff
42
43     class FusionSegmenter(TrajectoryHybridSegmenter):
44         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45         B)."""
46
47         family = "fusion"
48         display_label = "Fusion"
49
50         def __init__(self, db: Any, config: Any):
51             super().__init__(db, config)
52             # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53             default path).
54             self._person: Optional[Any] = None
55
56             @property
57             def name(self) -> str:
58                 return "fusion_hybrid"
59
60             @property
61             def description(self) -> str:
62                 return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
63                 rallies, "
64
65                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
66                 and the "
67
68                 "AI provider sets semantic boundaries.")
69
70             def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,

```

```

Any]]:
59     substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60     if substrate == "tracknet":
61         cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62         return TrackNetRunner(cfg), {
63             "weights_path": cfg.weights_path,
64             "queue_length": cfg.queue_length,
65             "fusion_substrate": "tracknet",
66         }
67     wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68     return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                               frame_width: float, video_path: str,
72                               idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
74 video_path, idx_cfg)
75         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
76         persisted
77         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
78         estimated
79         # over the whole trajectory by rally_gate (camera-agnostic).
80         gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,
81 min_crossings=0)
82         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
83
84         fcfg = idx_cfg.get("fusion", {})
85         if fcfg.get("cue_flags", {}).get("person", False):
86             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
87 fcfg))
88         return stats
89
90     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
91 frame_width: float, video_path: str,
92 fcfg: Dict[str, Any]) -> Dict[str, float]:
93         """GPU path (person cue ON): run the person detector over this window's
94 ORIGINAL-video
95 frame range and compute the scale/translation-invariant person features. Lazily
96 builds
97 ONE PersonDetector. ? real-video verification is owner-side; this wiring is
98 mock-tested."""
99         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
100 this module.
101         from backend.pipeline.detectors.person_detector import PersonDetector
102         if self._person is None:
103             self._person = PersonDetector(self.config)
104
105         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
106         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
107         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
108         fw = det["frame_width"] or frame_width
109         fh = det["frame_height"] or 0.0
110
111         poly = fcfg.get("court_polygon")
112         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
113         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
114         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
115         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
116 court_polygon=polygon, net=net)
117
118     def _select_for_handoff(self, windows: List[Dict[str, Any]],
119 window_stats: List[Dict[str, Any]],
120 idx_cfg: Dict[str, Any]) -> List[int]:
121         """Served Gate A/B over the (already-enriched) per-window stats. Default

```

```

thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126                 keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

4. user (2026-06-13T19:13:57.313Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Sequence, Tuple
24
25    Interval = Tuple[float, float]
26
27
28    @dataclass
29    class GatedWindow:
30        start: float
31        end: float
32        crossings: int
33        visible_points: int
34        kept: bool
35
36
37    def _spread(vals: Sequence[float]) -> float:
38        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39        if len(vals) < 2:
40            return 0.0
41        s = sorted(vals)

```

```

42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93     """Annotate each (start,end) window with its shuttle net-crossing count and a
94     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95     axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96     axis, net, span = estimate_play_axis(points)
97     deadband = deadband_frac * span
98     out: List[GatedWindow] = []
99     for (s, e) in windows:
100         wp = _points_in(points, fps, s, e)
101         c = count_net_crossings(wp, axis, net, deadband)
102         vis = sum(1 for p in wp if p.visible)
103         out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                               kept=c >= min_crossings))
105     return out

```



```

frames={cw['active_frame_count']} "
198             f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
199
200         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
201         # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
202         # persistence and the handoff gate below.
203         window_stats = [
204             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
205             for cw in all_candidate_windows
206         ]
207
208         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
209         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
210         if store_candidates:
211             for i, cw in enumerate(all_candidate_windows):
212                 candidate_ids[i] = self.db.add_candidate_segment(
213                     video_id, detector=self.name,
214                     start_time=cw["start"], end_time=cw["end"],
215                     chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
216                     stats=window_stats[i], detection_params=detection_params,
217                 )
218
219         if not all_candidate_windows:
220             return []
221
222         # --- Phase 3: AI provider handoff over padded candidate clips ---
223         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
224         # candidates above stay the full superset ? only the served play_segments are
gated.
225         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
226         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
227         os.makedirs(handoff_dir, exist_ok=True)
228         logger.info("Phase 3: %s validation on %d candidate windows...",
229                     provider_name, len(handoff_indices))
230
231         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
232             w_start = max(0, window["start"] - handoff_padding)
233             w_end = min(video_duration, window["end"] + handoff_padding)
234             w_dur = w_end - w_start
235             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
236             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
237                 return []
238             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
239             segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
240                                                  label=f"Handoff {wi+1}")
241             valid = []
242             for seg in segments:
243                 try:
244                     seg["start_time"] = float(seg["start_time"]) + w_start
245                     seg["end_time"] = float(seg["end_time"]) + w_start
246                     seg["_candidate_id"] = candidate_ids[wi]
247                     valid.append(seg)
248                 except (ValueError, TypeError, KeyError) as e:
249                     logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
250             try:
251                 os.remove(w_path)

```

```

252             except OSError:
253                 pass
254             return valid
255
256         ai_segments: List[dict] = []
257         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258             futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                 for wi in handoff_indices]
260             for future in futures:
261                 ai_segments.extend(future.result())
262             try:
263                 os.rmdir(handoff_dir)
264             except OSError:
265                 pass
266             logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268             # --- Phase 4: DB population + candidate linking ---
269             segments_data = []

```

7. assistant (2026-06-13T19:14:08.309Z)

Now let me confirm the claim by tracing the exact computational structure. Let me verify:

1. The list comprehension at line 203-206 calls `\_enrich\_candidate\_stats` once per candidate window (W times).
2. Each call to `\_enrich\_candidate\_stats` (fusion_hybrid.py:77) calls `rally\_gate.gate\_windows(points, [(cw["start"], cw["end"])], fps, min\_crossings=0)` with a single-window list.
3. `gate\_windows` (rally_gate.py:96) calls `estimate\_play\_axis(points)` over the FULL `points` list every time.
4. `estimate\_play\_axis` (rally_gate.py:61-66) does two list comprehensions over all points and two `\_spread` calls (each sorts ? $O(P \log P)$) plus `\_median` (sorts again ? $O(P \log P)$).

Let me verify there's no caching of axis/net anywhere and confirm `points` is the full trajectory passed unchanged.

8. user (2026-06-13T19:14:09.675Z)

```

backend\eval\distill_local.py:56: gated = rally_gate.gate_windows(points, intervals, fps,
min_crossings=0) # for net-crossings
backend\pipeline\detectors\rally_gate.py:55: def estimate_play_axis(points) -> Tuple[str, float,
float]:
backend\pipeline\detectors\rally_gate.py:91: def gate_windows(points, windows: List[Interval],
fps: float,
backend\pipeline\detectors\rally_gate.py:96:     axis, net, span = estimate_play_axis(points)
backend\pipeline\detectors\rally_gate.py:112:         gate_windows(points, windows, fps,
min_crossings, deadband_frac) if g.kept]
backend\pipeline\segmenters\fusion_hybrid.py:8:- ``_enrich_candidate_stats`` ? always adds the
**net-crossing count** (Gate A signal,
backend\pipeline\segmenters\fusion_hybrid.py:70:     def _enrich_candidate_stats(self, cw:
Dict[str, Any], points: List[Any], fps: float,
backend\pipeline\segmenters\fusion_hybrid.py:73:         stats =
super()._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
backend\pipeline\segmenters\fusion_hybrid.py:77:         gated = rally_gate.gate_windows(points,
[(cw["start"], cw["end"])], fps, min_crossings=0)
backend\pipeline\segmenters\trajectory_hybrid.py:82:     def _enrich_candidate_stats(self, cw:
Dict[str, Any], points: List[Any], fps: float,
backend\pipeline\segmenters\trajectory_hybrid.py:204:
self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)

```

9. assistant (2026-06-13T19:14:12.047Z)

The finding is confirmed. Let me also verify the base class `\_enrich\_candidate\_stats` to ensure the `points` passed is the full trajectory (not a per-window subset) and that no axis caching exists.

10. user (2026-06-13T19:14:13.039Z)

```
82         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
83                                     frame_width: float, video_path: str,
84                                     idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
85             """Stats dict persisted into `candidate_segments.stats` for one window. The
BASE
86             returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
87             rows; `FusionSegmenter` overrides this to add `net_crossings` (+ person-cue
88             features). `points` are the whole-video trajectory points
(TrajectoryPoint)."""
89             return {
90                 "peak_velocity": cw["peak_velocity"],
91                 "mean_velocity": cw["mean_velocity"],
92                 "active_frame_count": cw["active_frame_count"],
93                 "track_id_count": cw["track_id_count"],
94             }
95
96         def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                               window_stats: List[Dict[str, Any]],
98                               idx_cfg: Dict[str, Any]) -> List[int]:
99             """Indices of the candidate windows that proceed to the AI handoff (and thus may
100             become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101             `FusionSegmenter` overrides this to apply Gate A/B when thresholds are raised.
102             Persisted candidates are NOT affected ? they remain the full superset for
offline
103             re-scoring."""
104             return list(range(len(windows)))
105
106         def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                           player_pool: Optional[List[str]] = None,
108                           max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109             # override-ignore: the ABC declares the Result contract (Success|Failure)
110             # but every live segmenter still returns a bare list ? the documented
111             # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
```

11. assistant (2026-06-13T19:14:29.093Z)

Confirmed. The docstring at line 88 explicitly states "`points` are the whole-video trajectory points", confirming the full trajectory is passed to each per-window call.

The finding holds completely. Let me trace through the exact computational claim one final time:

- **trajectory_hybrid.py:203-206** ? list comprehension over `all\_candidate\_windows` (W windows), each calling `\_enrich\_candidate\_stats(cw, points, ...)` where `points` is the full whole-video trajectory.
- **fusion_hybrid.py:77** ? each call invokes `rally\_gate.gate\_windows(points, [(cw["start"], cw["end"])], fps, min\_crossings=0)` with the full `points` and a single-element window list.
- **rally_gate.py:96** ? `gate\_windows` calls `estimate\_play\_axis(points)` unconditionally at the top, over the full P-point trajectory.
- **rally_gate.py:61-66** ? `estimate\_play\_axis` builds two filtered lists over P points, then `\_spread(xs)` + `\_spread(ys)` (rally_gate.py:37-44, each sorts ? $O(P \log P)$) plus two `\_median` calls (rally_gate.py:47-52, each sorts ? $O(P \log P)$).

So for W windows the axis/net estimate is recomputed W times over the full P-point trajectory: $O(W \cdot P \log P)$. And the comment at rally_gate.py:95 explicitly documents axis/net as "estimated once over the whole trajectory (stable)" ? window-independent ? confirming the redundancy is real, not a per-window-dependent computation.

The severity assessment is fair: it's genuinely wasteful and redundant, but it is pure-Python,

GPU-free, and `FusionSegmenter` is default-OFF (opt-in, not the default segmenter). The per-window `\_points\_in` scan (rally_gate.py:88, $O(P)$ over the full list, also redundant since it could index a sorted-by-frame structure) is unavoidable per-window work but is not the headline cost. The redundant part is specifically the `estimate\_play\_axis` re-estimation. Low severity is correct.

The fix suggestion is also valid and matches the code: estimate axis/net/deadband once per video, then either call `count\_net\_crossings` per window directly (the existing public function at rally_gate.py:69) or call `gate\_windows(points, all\_windows, ...)` a single time and index by window.

12. user (2026-06-13T19:14:44.325Z)

Structured output provided successfully

13. assistant (2026-06-13T19:14:49.615Z)

Verified the finding against the actual code and confirmed it as real (low severity). The redundant-work chain holds exactly as described: the per-window comprehension at trajectory_hybrid.py:203-206 calls into fusion_hybrid.py:77, which invokes `rally\_gate.gate\_windows` once per window with a singleton list, and that function unconditionally re-runs `estimate\_play\_axis(points)` (two $O(P \log P)$ sorts + a median sort) over the full trajectory on every call ? despite rally_gate.py:95 documenting axis/net as "estimated once over the whole trajectory (stable)." Result is $O(W \cdot P \log P)$ of redundant work. Severity stays low because the path is pure-Python/GPU-free and `FusionSegmenter` is default-OFF. The structured output has been submitted.